

Rapport de Projet de Programmation

Développement du Jeu Traditionnel Songo

Présenté par :

LEUDJEU NJIATCHA YANIS AEL

Cours :

INF222 - DEVELOPPEMENT WEB

Ce rapport documente la conception, l'algorithmique et l'organisation du code pour les deux versions (Locale et Multijoueur Client-Serveur) du jeu Songo.

Année Académique 2025-2026

1 Introduction au Songo et Contexte du Projet

Le Songo est un jeu de stratégie abstrait traditionnel profondément ancré dans la culture, faisant partie de la grande famille des jeux de Mancala (jeux de semailles). La transcription d'un tel jeu dans un environnement numérique nécessite une compréhension absolue de ses règles complexes : la rotation anti-horaire, la capture conditionnelle, et surtout les contraintes de solidarité (ne pas affamer son adversaire).

Le but de ce projet universitaire était de réaliser une numérisation complète de ce jeu en deux étapes majeures :

- **La Version 1 (V1)** : Une version jouable localement, sur un seul écran, permettant de valider l'interface visuelle et la logique algorithmique (le moteur du jeu).
- **La Version 2 (V2)** : Une évolution majeure vers une architecture réseau, où deux joueurs distants s'affrontent de manière asynchrone via des requêtes AJAX et un serveur PHP.

Ce rapport détaillera méticuleusement l'architecture logicielle, les choix algorithmiques et l'organisation du code source qui ont permis de mener à bien ce projet.

2 Architecture et Organisation du Code - Version 1 (Locale)

La Version 1 repose sur un socle technique classique du développement front-end moderne : HTML5(Index.html), CSS3(Index.css) et JavaScript (Songo.js). Afin de garantir un code maintenable et évolutif, le principe de séparation des préoccupations a été strictement appliqué.

2.1 Modélisation de l'état : Le Constructeur

Toute l'intelligence du jeu a été encapsulée dans une classe JavaScript nommée *Songo*. L'état du plateau, les scores, et les métadonnées du tour y sont gérés.

Voici l'initialisation de cette classe, qui définit les bases de notre jeu :

```
1 export class Songo {  
2   constructor(){  
3     this.coteJoueur1 = [5,5,5,5,5,5,5];  
4     this.coteJoueur2 = [5,5,5,5,5,5,5];  
5     this.pointJoueur1 = 0;  
6     this.pointJoueur2 = 0;  
7  
8     this.tour = 1;  
9     this.gameOver = false;
```

```
10     this.winner = null;
11     this.lastPion = -1;
12     this.lastSide = -1;
13     this.caseCapter = [];
14 }
15 }
```

Listing 1 – Initialisation de la classe Songo

Analyse algorithmique : Les tableaux `coteJoueur1` et `coteJoueur2` représentent physiquement le plateau de jeu. L'utilisation de tableaux à une dimension de taille 7 (initialisés avec 5 graines) simplifie grandement les calculs d'index lors des itérations. Les variables `lastPion` et `lastSide` sont cruciales, car elles représentent la mémoire de l'algorithme : elles indiquent où le dernier pion est tombé, ce qui est la condition *sine qua non* pour évaluer une éventuelle capture.

2.2 La dynamique du jeu : L'algorithme de distribution

La distribution (ou semaille) est le cœur mécanique du Songo. Le défi majeur était de gérer la rotation continue (le passage d'un camp à l'autre) ainsi que la règle spécifique qui impose de sauter la case de départ si le nombre de graines permet d'effectuer un tour complet.

```
1 distribuerPions(idJ, choix){
2     let tab = idJ === 1 ? this.coteJoueur1 : this.coteJoueur2;
3     // [...] (verifications de base omises pour concision)
4
5     let index = choix;
6     let id = idJ;
7     let pion = tab[choix];
8     tab[choix] = 0; // On vide la case choisie
9
10    const caseDepartSide = idJ;
11    const caseDepartIndex = choix;
12    this.cheminDistribution = [];
13
14    while(pion > 0){
15        index--; // Deplacement de la droite vers la gauche
16
17        if (index < 0) { // Changement de camp
18            id = id === 1 ? 2 : 1;
19            tab = id === 1 ? this.coteJoueur1 : this.coteJoueur2;
20            index = 6;
21        }
22
23        // Saut de la case de depart (tour complet)
24        if (id === caseDepartSide && index === caseDepartIndex) {
```

```
25         continue;
26     }
27
28     tab[index]++;
29     pion--;
30
31     // Enregistrement pour l'animation visuelle
32     this.cheminDistribution.push({cote: id, index: index});
33
34     this.lastPion = index;
35     this.lastSide = id;
36 }
37 return true;
38 }
```

Listing 2 – Algorithme de distribution des graines

Explication détaillée : L'algorithme décrémente l'index pour symboliser un déplacement anti-horaire (de la droite vers la gauche sur le tableau du joueur). Lorsque l'index devient inférieur à zéro, une permutation mathématique et logique s'opère : le pointeur change de tableau (du camp 1 vers le camp 2 ou inversement) et l'index est réinitialisé à 6 (la droite de l'autre camp). La boucle **while** garantit que chaque graine est traitée individuellement. Enfin, le tableau **cheminDistribution** stocke chaque coordonnée visitée, ce qui permet au fichier **main.js** d'animer le déplacement des graines de manière asynchrone sur l'interface graphique.

2.3 La logique de Capture

La capture dans le Songo obéit à des règles extrêmement strictes : elle ne peut se produire que si la dernière graine tombe dans le camp adverse, et uniquement si cette case (ainsi que les précédentes ininterrompues) contient 2, 3 ou 4 graines. Une subtilité importante a été implémentée : l'index 0 du camp adverse ne peut jamais être capturé.

```
1 capturePions(idJ){
2     this.caseCapter = [];
3     if (this.lastSide !== idJ) { // Verifie si on est chez l'
        adversaire
4         let tabAdverse = this.lastSide === 1 ? this.coteJoueur1 : this
            .coteJoueur2;
5         let currentIndex = this.lastPion;
6         let capturedThisTurn = 0;
7
8         while (currentIndex <= 6) {
9             // R g le d'or : interdiction de capturer l'index 0
10            if (currentIndex === 0) { break; }
11        }
```

```

12         // Condition de capture (2, 3 ou 4 graines)
13         if (tabAdverse[currentIndex] >= 2 && tabAdverse[
currentIndex] <= 4) {
14             capturedThisTurn += tabAdverse[currentIndex];
15             this.caseCapter.push(currentIndex);
16             tabAdverse[currentIndex] = 0;
17             currentIndex++; // On remonte vers la droite
18         } else {
19             break; // Rupture de la chaine
20         }
21     }
22
23     if (capturedThisTurn > 0) {
24         if (idJ === 1) this.pointJoueur1 += capturedThisTurn;
25         else this.pointJoueur2 += capturedThisTurn;
26     }
27 }
28 }

```

Listing 3 – Algorithme de capture

Justification des choix : La boucle parcourt le tableau en sens inverse (incrément de `currentIndex`) pour vérifier les cases précédentes, reproduisant ainsi fidèlement le chaînage des captures propre aux règles réelles du jeu. Dès qu'une case ne respecte pas la condition numérique ou dès que la case "sanctuaire" (index 0) est atteinte, la directive `break` interrompt immédiatement le processus.

2.4 La règle de Solidarité (Nourrir l'adversaire)

Il est interdit d'affamer son adversaire. Si ce dernier n'a plus de graines, le joueur actuel a l'obligation de lui en fournir.

```

1 peutNourrir(idJ){
2     const tab = idJ === 1 ? this.coteJoueur1 : this.coteJoueur2;
3     for(let i = 0; i < 7; i++){
4         // S'il y a plus de graines que la distance pour atteindre l'
adversaire
5         if(tab[i] > i){
6             return true;
7         }
8     }
9     return false;
10 }

```

Listing 4 – Vérification de la capacité à nourrir l'adversaire

Cette méthode est élégante par sa simplicité : la distance entre une case d'index `i` et le camp adverse est exactement égale à `i`. Il suffit donc que la case contienne `i + 1`

graines (soit `tab[i] > i`) pour que l'une d'elles bascule de l'autre côté du plateau.

3 Architecture et Organisation du Code - Version 2 (Multijoueur Client-Serveur)

La grande évolution de ce projet a été le passage d'un modèle local à un modèle distribué. Le défi n'était plus seulement de calculer les règles, mais d'assurer que deux navigateurs distants partagent une vue strictement identique et synchronisée de l'état de la partie.

3.1 Le modèle Client-Serveur

Nous avons opté pour une pile technologique XAMPP intégrant un serveur web (Apache) et PHP. La logique de jeu reste gérée majoritairement par le client (dans la classe `Songo`), ce qui a permis de réutiliser 100% du code de la V1. Le rôle du serveur PHP se limite à être l'arbitre et le conservateur des données.

3.2 La gestion de l'état (`State.json`)

Au lieu de déployer une base de données complexe comme MySQL, nous avons mis en place un fichier texte `state.json`. Le PHP s'occupe de lire et d'écrire dans ce fichier avec des verrous de concurrence.

```
1 toJSON() {
2     return {
3         coteJoueur1: [...this.coteJoueur1],
4         coteJoueur2: [...this.coteJoueur2],
5         pointJoueur1: this.pointJoueur1,
6         pointJoueur2: this.pointJoueur2,
7         tour: this.tour,
8         gameOver: this.gameOver,
9         winner: this.winner,
10        capturedCells: [...this.caseCapter]
11    }
12 }
```

Listing 5 – Méthode de sérialisation JSON de l'état

Cette méthode extrait l'intégralité du "cerveau" de l'objet de jeu. Les opérateurs de décomposition (spread operator `...`) garantissent une copie par valeur et non par référence des tableaux.

3.3 Synchronisation via requêtes asynchrones

La fluidité de la V2 repose sur l'utilisation de la Fetch API de JavaScript. Pour simuler le temps réel, nous avons implémenté une technique de "polling" (sondage régulier).

[Insérez ici vos explications sur comment `main.js` ou `script.js` fait des requêtes `fetch()` toutes les X secondes pour vérifier si c'est son tour. Vous pouvez y joindre une capture d'écran de l'onglet "Network" (Réseau) du navigateur montrant les requêtes AJAX successives.]

4 Conception Visuelle et Interface Utilisateur (UI/UX)

Le succès d'un jeu vidéo réside autant dans sa fiabilité mathématique que dans son rendu esthétique. Pour ce projet, le CSS a été écrit de zéro (Vanilla CSS) pour offrir un rendu luxueux, inspiré du bois naturel des plateaux de Songo traditionnels.

[Insérez ici des captures d'écran de votre plateau (V1 et V2).]

- **Design Responsive** : L'utilisation de `flexbox` et `grid` permet au plateau de s'adapter aux différents écrans.
- **Feedbacks visuels** : Les cases capturées s'illuminent, offrant un retour visuel clair au joueur sur le résultat de son action.
- **Écran d'attente V2** : Un overlay semi-transparent s'affiche lorsque ce n'est pas le tour du joueur, désactivant ainsi toute interaction avec le DOM et prévenant la corruption des requêtes serveur.

5 Difficultés Techniques Rencontrées et Solutions Apportées

5.1 Problèmes liés aux permissions fichiers (Serveur local)

Lors du développement de la V2 sur XAMPP, une erreur récurrente bloquait la sauvegarde de la partie : le fichier `state.json` refusait les écritures PHP à cause de restrictions du système d'exploitation. La solution a consisté à modifier les droits d'accès du dossier du projet (configuration des permissions en écriture).

5.2 Gestion de l'asynchronisme visuel

L'un des plus grands défis en JavaScript a été de synchroniser le calcul mathématique, qui est instantané, avec l'animation visuelle des graines, qui prend plusieurs se-

condes. La solution apportée a été l'enregistrement préalable du `cheminDistribution` par l'algorithme, puis la lecture asynchrone de ce chemin dans le DOM via des appels successifs à `setTimeout`.

6 Conclusion

Le développement de ce jeu de Songo a été un exercice de programmation complet et exigeant. La Version 1 nous a forcés à modéliser des règles mathématiques strictes (distribution anti-horaire, solidarité, sauts) dans un langage orienté objet. La Version 2 nous a propulsés dans les problématiques des architectures distribuées (Client-Serveur, formats JSON, requêtes HTTP, droits d'écriture PHP).

Le résultat est un produit fonctionnel, dont le code est modulaire, maintenable, et séparé en couches logiques distinctes (Modèle pour l'état, Vue pour le HTML/CSS, Contrôleur pour le `main.js`). Des améliorations futures pourraient inclure l'intégration d'une intelligence artificielle (algorithme Min-Max) pour jouer seul, ou le passage aux WebSockets pour un mode multijoueur en temps réel natif plutôt que par requêtes AJAX périodiques.